ADVANCED DATA MANAGEMENT · Final Exam Project · Università degli Studi di Trieste

ThreatGraph: FAIR-Oriented Data Management of Exploited Vulnerabilities

An OSINT case study on linking open, traceable and versioned cyber-threat data



UNIVERSITY OF TRIESTE

Tavano Matteo · SM3800057

Main Research Question

“How can heterogeneous open data sources be integrated into one database so that every link between them is traceable, versioned and reusable?”

OSINT

Open Source Intelligence: Information obtained from publicly available sources. Here “open source” means public source, not free software. Sources used in this project are public, freely accessible and machine-readable.

Cyber threat

A weakness in software, identified by a CVE code, that attackers are confirmed to have used in real attacks. A vulnerability is the weakness; a threat is someone using it.

Project Overview: The Four Open Sources

Four public sources describe different aspects of the same exploited vulnerabilities. They are linked through shared identifiers, and every link is recorded together with its source, file and version.

NVD

The registry of vulnerabilities

A database of NIST (USA). NVD enriches CVE records with severity scores and weakness types.

KEV

The list of vulnerabilities used in attacks

A curated list by CISA (USA). It holds only CVEs that confirmed to have been exploited.

ATT&CK

The knowledge base of attacker moves

A MITRE catalogue. A tactic is the goal (why). A technique is the method (how).

PKEV

The bridge built by experts

The Center for Threat-Informed Defense linked some KEV CVEs to ATT&CK techniques, manually, as expert analysis.

Key point. Only PKEV links a vulnerability to an ATT&CK technique, and only for some vulnerabilities. This link is an expert's claim, not a fact, so each mapping is stored with its source, file, version and evidence. This is why provenance matters.

Project in Numbers

1,391
exploited vulnerabilities (KEV), all in NVD

419
vulnerabilities with an ATT&CK mapping

30.1%
of KEV are mapped

1,183
mappings between vulnerabilities and techniques

155
different techniques used

4
open data sources

Key point. The value of the project is traceability: every number and every mapping can be followed back to a file, a version and a SHA-256 hash

Project Outline

1
Acquire
download pinned files, hash them, write the manifest

2
Clean
validate, type, flag anomalies

3
Connect
load into Django, link by keys

4
Publish
REST API and JSON-LD export

5
Archive
hashed snapshots of every version

Raw Data: Sources

Source	Publisher	Format	Pinned version	Content	Licence
KEV	CISA, GitHub mirror	JSON	2025-07-28	1,391 entries	CC0 1.0
NVD	NIST, via fkie-cad mirror	One JSON per CVE	2025-07-28	1,391 records	NVD terms of use
ATT&CK	MITRE	STIX 2.1, 46 MB	Enterprise 16.1	799 techniques, 14 tactics	MITRE terms of use
PKEV	Center for Threat-Informed Defense	JSON	2025-07-28	1,183 mappings	Apache 2.0 (repository)

Raw Data: Provenance

Pinned
Every URL points to an immutable commit or tag, never to a branch that can move.

Hashed
A SHA-256 is stored for every file. A mismatch means the source changed or the transfer broke and the download fails.

Immutable
The raw folder is never modified. Later steps write their results in other folders.

Anomalies found in the sources, kept and reported:

15 KEV entries have a due date before the date added. They are flagged in the relational tables.

2 ATT&CK relationship objects were created after their modified date.

Data Cleaning Rules

Source	Rules, in brief
KEV	Ransomware “unknown” is not “false” (1,100 unknown). A due date before the added date is kept and flagged (15 entries).
NVD	One **CVSS v3.1** score per CVE: the NVD score, otherwise the first one from the CNA (135 CVEs). Uninformative weaknesses are kept and flagged (373 rows).
ATT&CK	Only techniques and tactics are kept (799 + 14). Revoked and deprecated techniques are kept with a status (131 + 12).
PKEV	Dirty references are split or flagged (18 split, 25 flagged). No row means “not mapped”, never “no technique” (419 of 1,391 mapped).

General rules:

Raw files are never modified, and their hashes are checked before cleaning. Missing data stay missing: blanks become NULL and nothing is imputed or copied from another source. Every rule is recorded in the cleaning manifest.

Data Cleaning: Relational Tables

Table	Source	Rows	Primary Key	Description
kev_vulnerability	KEV	1,391	cve_id	Exploited vulnerabilities
kev_cwe	KEV	1,328	(cve_id, cwe_id)	Weakness types listed by KEV for each CVE
nvd_cve	NVD	1,391	cve_id	CVE descriptions and CVSS scores
nvd_cve_cwe	NVD	2,523	(cve_id, cwe_id, source)	Weakness types listed by NVD for each CVE
attack_tactic	ATT&CK	14	tactic_id	Attacker goals (tactics)
attack_technique	ATT&CK	799	technique_id	Attacker methods (techniques and sub-techniques)
attack_technique_tactic	ATT&CK	1,051	(technique_id, tactic_id)	Links between techniques and tactics
pkev_mapping	PKEV	1,183	mapping_id	Expert mappings from CVEs to techniques
pkev_mapping_reference	PKEV	2,256	(mapping_id, reference)	Evidence links (URLs) for each mapping

Linking Data: Joins

A join connects two tables through a shared key. After cleaning, each join is checked: every key must find its match.

Join	Key	Relationship	Check result
KEV with **NVD**	cve_id	one-to-one	All 1,391 KEV entries have exactly one NVD record.
PKEV with **KEV**	cve_id	many-to-one	All 419 mapped CVEs are KEV entries.
PKEV with **ATT&CK**	technique_id	many-to-one	All 155 mapped techniques exist in ATT&CK 16.1.
ATT&CK technique with tactic	tactic_shortname	many-to-many	A technique can belong to several tactics: 1,051 links for 799 techniques.
KEV / **NVD** with **CWE**	cwe_id	many-to-many	1,188 CVEs have an informative NVD weakness.

Key point

The four sources describe different aspects of the same vulnerabilities. They are connected by shared identifiers (CVE id, technique id). The joins are checked: keys must find their match (**referential integrity**) and no record is lost.

One real vulnerability through the four sources: CVE-2024-34102

KEV

cveID
CVE-2024-34102

vendorProject
Adobe - Commerce and Magento

dateAdded
2024-07-17

knownRansomwareCampaignUse
Unknown

NVD

id
CVE-2024-34102

CVSS 3.1
9.8 CRITICAL

weakness
CWE-611 (XXE)

PKEV

capability_id
CVE-2024-34102

exploitation_technique
T1190

primary_impact
T1059

secondary_impact
T1005

ATT&CK

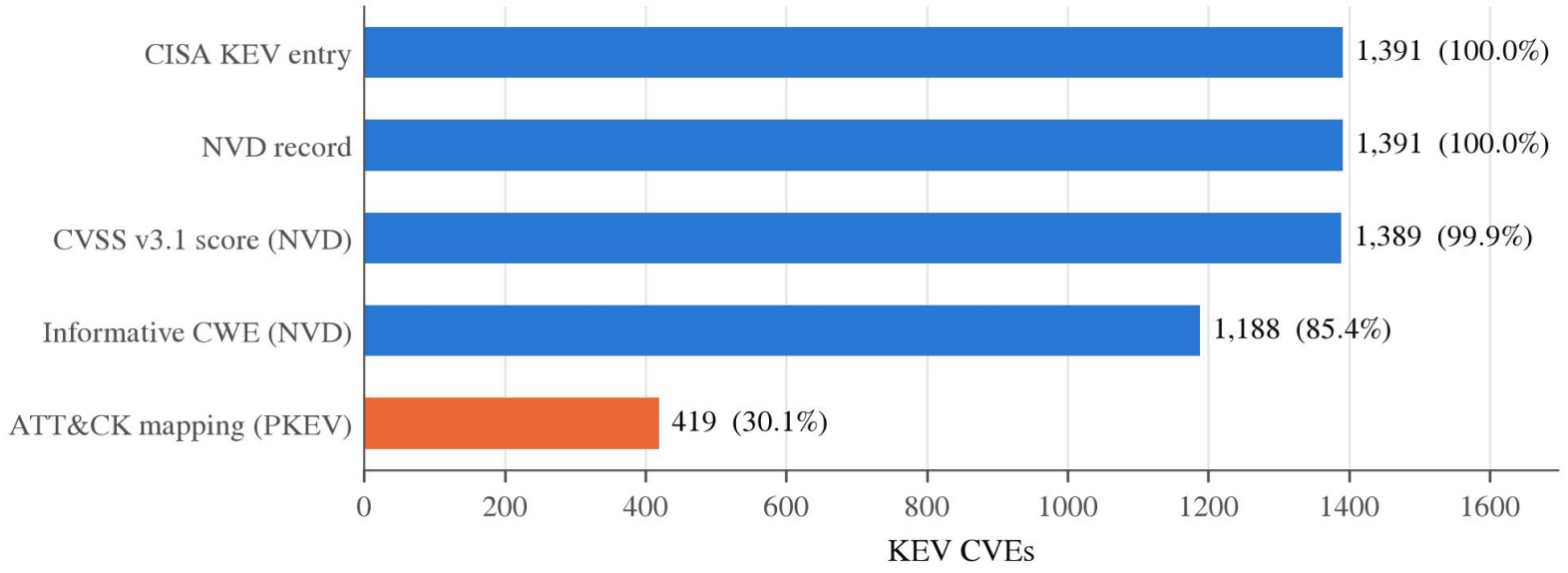
T1190
Exploit Public-Facing Application · **initial access**

T1059
Command and Scripting Interpreter · **execution**

T1005
Data from Local System · **collection**

Result: One vulnerability that is really exploited, with three attacker moves and their tactics. Each link records its source and its evidence. The yellow boxes are the join keys: the **CVE id** joins KEV, NVD and PKEV; the **technique id** joins PKEV and ATT&CK; the **tactic** comes from the technique.

Data Coverage



30.1%

419 of 1,391 KEV CVEs have a PKEV mapping

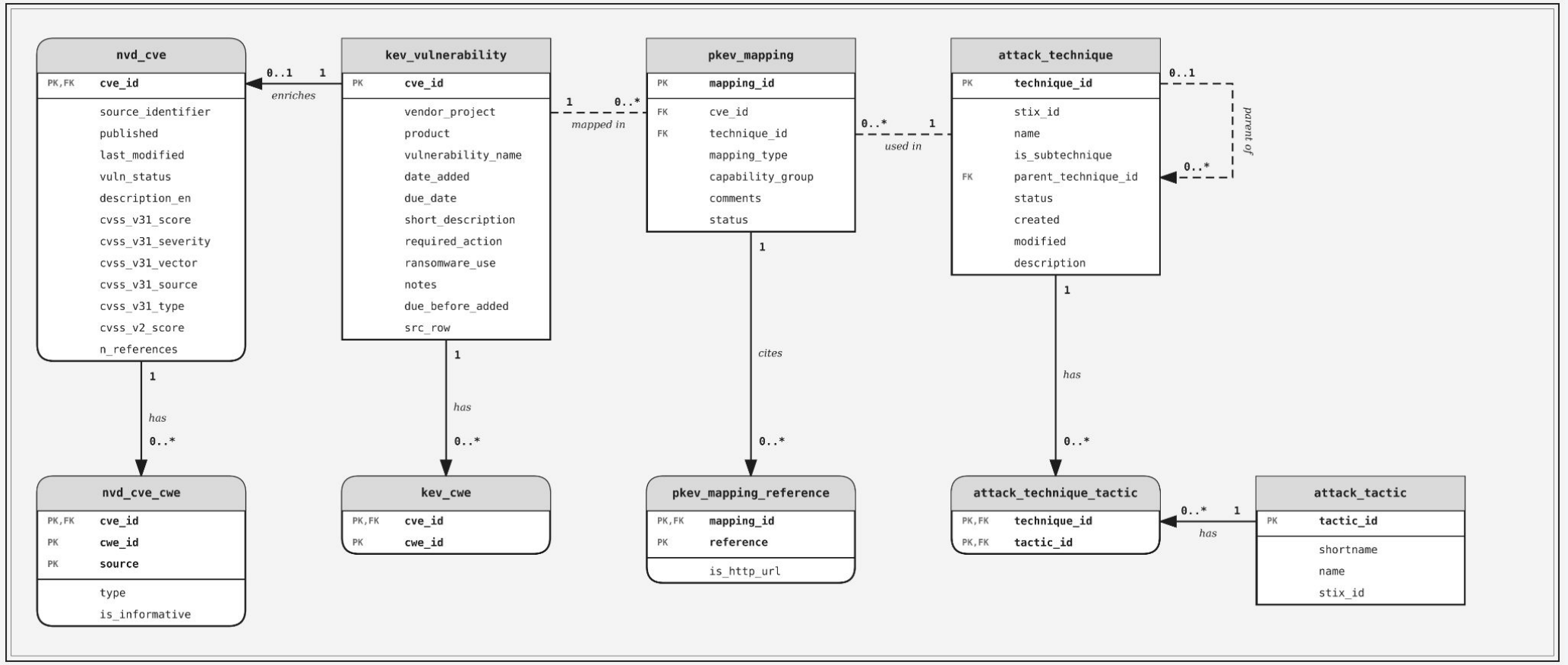
85.4%

have an informative CWE: *NVD-CWE-noinfo* and *NVD-CWE-Other* are not informative

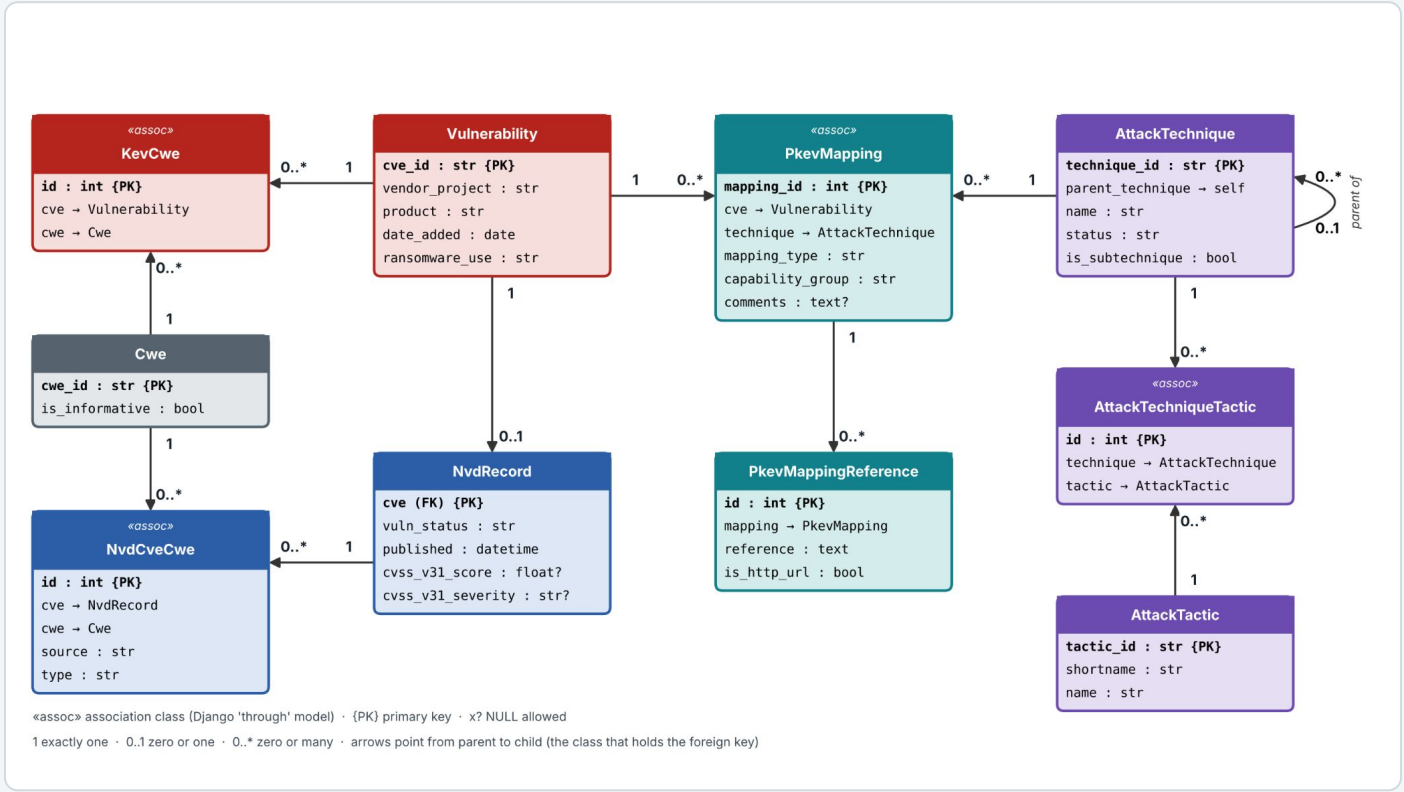
Key insight: KEV and NVD are fully joined, and the expert bridge exists for about a third of KEV.

Pay attention! The graph does not say that 70% of CVEs have no technique. It only says that PKEV has not mapped them.

Logical Schema: Tables and Relationships



UML Class Diagram of the Django Models



generated by make_uml.py from catalog/models.py

Association class

PkevMapping links vulnerabilities and techniques and stores its own data: the mapping type, the group and the comments.

Keys

cve_id and technique_id come from the sources, so tables from different sources can be joined directly.

CWE class

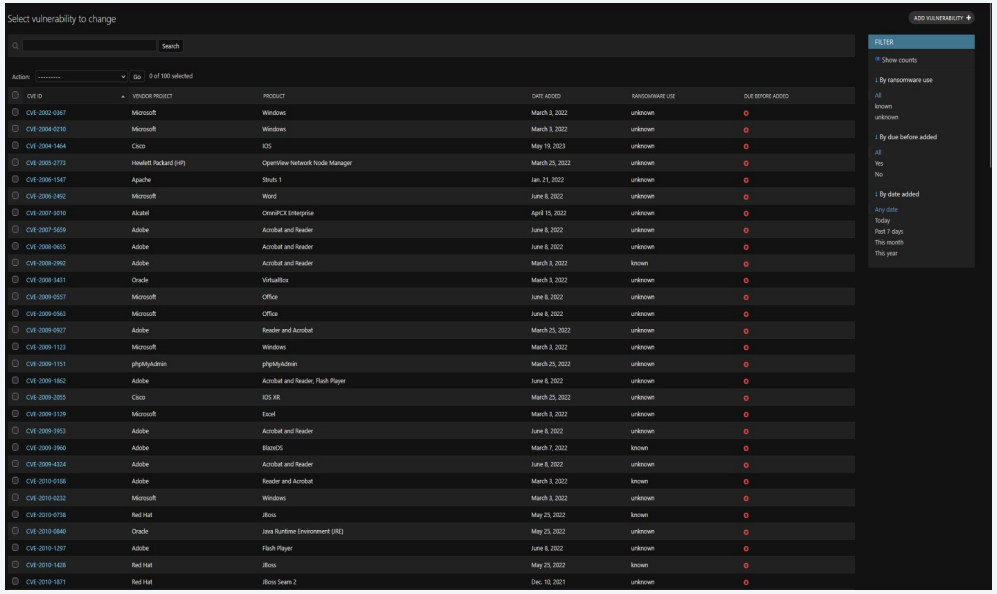
Cwe is derived from the weakness ids found in KEV and NVD.

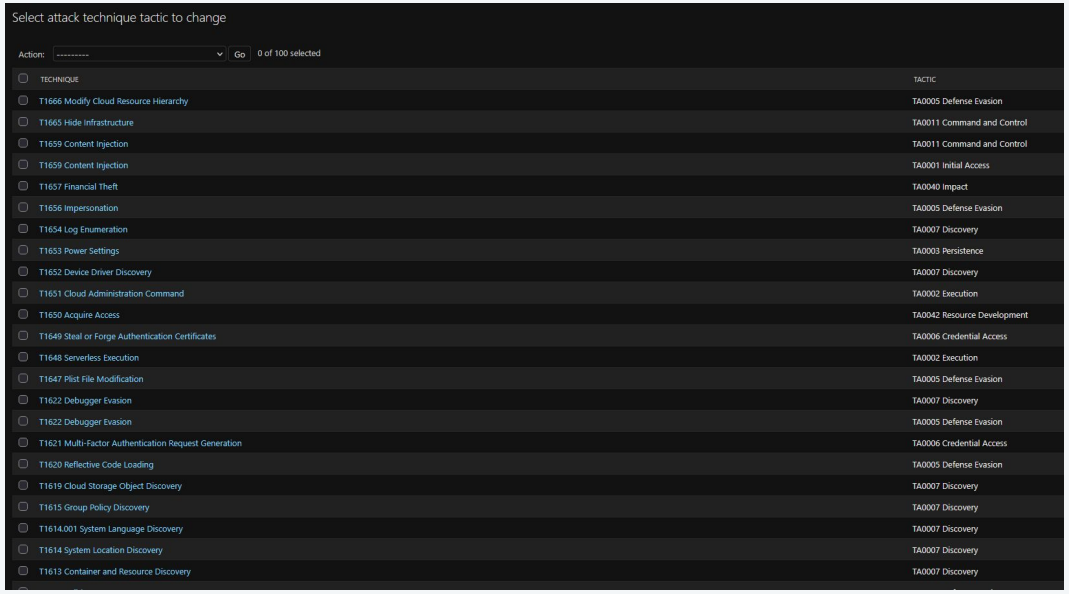
From design to code: Translation

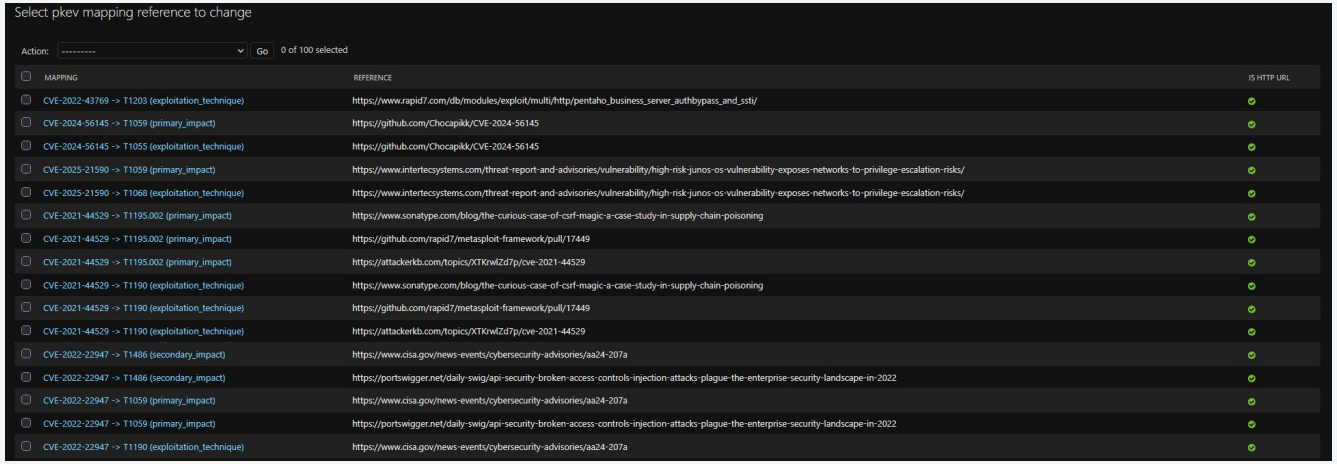
Each concept of the data model is written as a Django construct. The ORM then creates the tables from these models and maps rows to Python objects.

UML concept	In Django	Example in ThreatGraph
Class	class X(models.Model), one table	10 models: the 9 clean tables and Cwe (derived)
Attribute	Field: CharField, DateField, TextField	date_added is a DateField
Optional attribute (x?)	null=True	NULL means source did not say
One to many	ForeignKey	A PkevMapping points to one Vulnerability
Many to many	ManyToManyField with through	Vulnerability and Cwe via KevCwe
One to one (0..1)	OneToOneField	A Vulnerability has at most one NvdRecord
Association class	The through model with its own fields	PkevMapping
Self reference	ForeignKey("self")	parent_technique links sub-techniques to their parents
Link-table key	Surrogate id + UniqueConstraint	(cve, technique, mapping_type) is unique

Django Demo







Commands

```
> python webapp/manage.py migrate
> python webapp/manage.py load_clean_data
> python webapp/manage.py runserver
```

Django: Integrity and Tools

Constraints
The database rejects rows that break the rules: unique keys, valid values and no links to missing records.

One command loads Data
One command loads all tables in a single transaction: all or nothing. Running it twice gives the same database.

Round-trip export
Exporting the tables back to CSV gives the same SHA-256 as the clean files: nothing is lost or changed.

Built-in tools
The Django admin lets users browse and filter the data. A /stats/ page and a page for each vulnerability are available.

SQLite is enough here: the data is loaded once and then only read, by one user. If many users had to write at the same time, a server database such as PostgreSQL would be needed.

OAIS: Submit, Store, Deliver

Submission

Producers (SIP)

CISA, NIST, MITRE and CTID publish the four sources. The downloaded files are the SIPs, the **Submission Information Packages**: what arrives at the archive.

Archival

Ingest (SIP to AIP)

Validate, clean, load and create the snapshot. The preserved result is the AIP, the **Archival Information Package**: data, provenance and checksums.

Dissemination

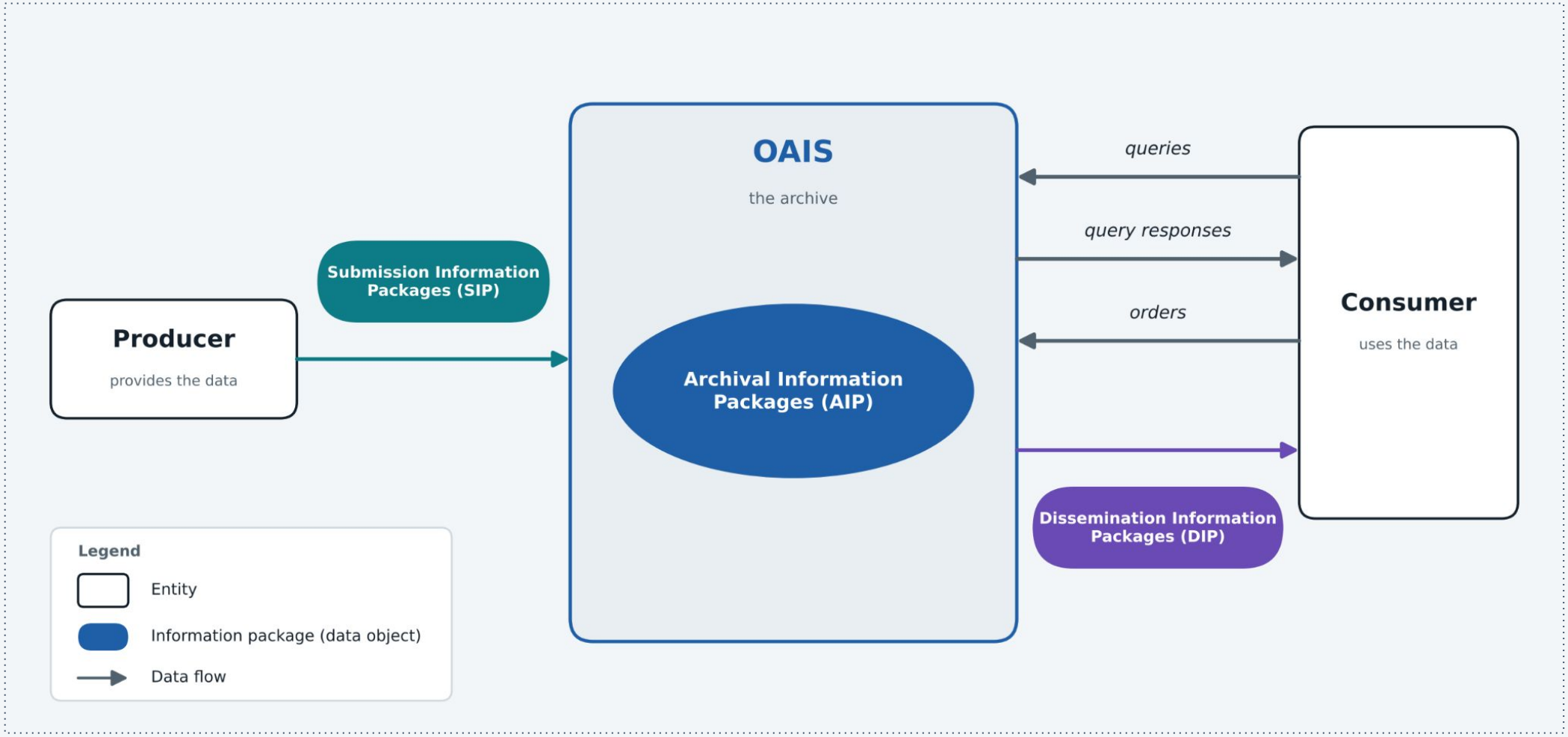
Access (DIP)

On request, the API and the JSON-LD export deliver DIPs, the **Dissemination Information Packages**: what reaches the people who use the data.

OAIS (ISO 14721:2012) is used as a reference model. Four sources that become one snapshot are the "*many SIPs, one AIP*" case. Each new download produces a new AIP, linked to the previous one.

> **Preservation planning** is not implemented: open formats and checksums only make a future migration easier.

The OAIS scheme



Project pipeline seen through OAIS

OAIS concept	In ThreatGraph
Producer	CISA, NIST, MITRE and CTID publish the data
SIP	The four pinned raw sources plus the manifest with SHA-256 hashes
Ingest	Validation, cleaning, loading and snapshot creation
AIP	The snapshot, read-only: CSV tables, manifest with hashes and content_id, JSON-LD description
Archival storage	data/snapshots/, append-only, checked by fixity (hashes)
Data management	The snapshot manifest and the DCAT / PROV description; content_id identifies each version
Access	The REST API, from the Django database
DIP	The JSON and JSON-LD documents returned by the API
Consumer	Analysts, notebooks, other teams
Not covered	Preservation planning (format migration). Open formats and checksum only make a future migration easier

A read-only REST API

Endpoints

```
/api/vulnerabilities/
/api/techniques/
/api/tactics/
/api/cwes/
/api/dataset/   (DCAT description of the dataset)
/api/context/   (JSON-LD vocabulary)
```

Example Query

```
/api/vulnerabilities/
  ?technique=T1190
  &severity=CRITICAL
  &ransomware_use=known
```

One call filters on KEV, NVD and ATT&CK fields.

A real export of CVE-2024-34102

```
{
  "@id": "https://nvd.nist.gov/vuln/detail/CVE-2024-34102",
  "@type": "Vulnerability",
  "cveId": "CVE-2024-34102",
  "vendorProject": "Adobe",
  "ransomwareUse": "unknown",
  "weakness": [{ "@id":
  "https://cwe.mitre.org/data/definitions/611.html",
      "cweId": "CWE-611" }],
  "nvd": { "cvssV31": { "score": 9.8,
      "severity": "CRITICAL" } }
}
```

The @id is a public IRI

NVD, MITRE and CWE addresses. Anyone can link to the same things.

Shared vocabularies

The context maps some terms to DCAT, PROV and Dublin Core; the others use ThreatGraph namespace.

Open point. Vocabulary namespace is a placeholder (example.org). A real, persistent one is still to be chosen.

Interoperability Example

Scenario. A company has open security incidents. Which of the exploited vulnerabilities behind them **should be fixed first**?

Roles

- **ThreatGraph**: JSON-LD from the API. Product, ransomware use, CVSS score, ATT&CK tactic.
- **SOC team**: JSON-LD with its own terms. 24 reported incidents each linked to a CVE.
- **Patch office**: a plain CSV. Owner and planned date of each fix.

Meeting point

- Each team keeps its own format and its own terms.
- One shared rule: a CVE is identified by its public address (e.g. https://nvd.nist.gov/vuln/detail/CVE-...)
- The three files are loaded together and one query is asked without conversion table.

The SOC log has 24 incidents and the patch tracker 24 fixes. 14 CVEs are in both, 10 only in the SOC log, 10 only in the patch tracker.

Merging the sources: Open incidents

Merge: Fetch Threat Graph **(JSON-LD)** from the **API**, only for the CVEs that the two teams mention.
The three documents are loaded into one RDF dataset, each in its own named graph.

Query 1: *How dangerous is the exploited vulnerability behind an open incident, and which ATT&CK tactic does it serve?*
It combines the SOC log (status, incident), ThreatGraph (product, ransomware use, CVSS) and ATT&CK (tactics, through PKEV mappings).

```
SELECT ?incident ?cve ?product ?ransomware ?score
       (GROUP_CONCAT(DISTINCT ?tactic; separator=", ") AS ?tactics)
WHERE {
  ?i a soc:Incident ; soc:status "open" ; soc:exploits ?v .          # SOC incident log
  ?v dct:identifier ?cve ; tg:product ?product ;                    # ThreatGraph
     tg:ransomwareUse ?ransomware ;
     tg:nvdRecord/tg:cvssV31/tg:baseScore ?score .
  OPTIONAL { ?v tg:hasMapping/tg:technique/tg:tactic/tg:shortname ?tactic }
  BIND(REPLACE(STR(?i), "^.*:", "") AS ?incident)
}
GROUP BY ?incident ?cve ?product ?ransomware ?score
ORDER BY DESC(?score) ?incident
```

Finds the open incidents in the SOC log and for each exploited CVE, adds from ThreatGraph the product, ransomware use, CVSS score and ATT&CK tactics.
Two sources (ThreatGraph and SOC) meet on the CVE address **?v**.

Query 1 Results: Open Incidents by Risk

Incident	CVE	Product	Ransomware	CVSS	ATT&CK tactics
INC-002	CVE-2018-4939	ColdFusion	unknown	9.8	initial-access, execution, persistence
INC-003	CVE-2011-2462	Acrobat and Reader	unknown	9.8	execution
INC-008	CVE-2019-19781	Application Delivery Controller (ADC), Gateway, and SD-WAN WANOP Appliance	known	9.8	initial-access, execution, persistence, discovery
INC-012	CVE-2023-48788	FortiClient EMS	known	9.8	initial-access, execution, command-and-control
INC-014	CVE-2024-21413	Office Outlook	unknown	9.8	initial-access, execution
INC-018	CVE-2017-5638	Struts	known	9.8	initial-access, execution, collection
INC-021	CVE-2023-36845	Junos OS	unknown	9.8	initial-access, execution
INC-005	CVE-2021-37975	Chromium V8	unknown	8.8	execution
INC-006	CVE-2021-28550	Acrobat and Reader	unknown	8.8	execution
INC-024	CVE-2023-7024	Chromium WebRTC	unknown	8.8	initial-access, persistence, privilege-escalation, defense-evasion
INC-023	CVE-2012-0754	Flash Player	unknown	8.1	execution, command-and-control
INC-009	CVE-2025-24993	Windows	unknown	7.8	execution, privilege-escalation, defense-evasion, impact
INC-017	CVE-2012-1535	Flash Player	unknown	7.8	execution, command-and-control
INC-020	CVE-2021-27065	Exchange Server	known	7.8	initial-access, persistence
INC-011	CVE-2024-57727	SimpleHelp	known	7.5	initial-access, execution, credential-access
INC-015	CVE-2023-0669	GoAnywhere MFT	known	7.2	initial-access, lateral-movement, impact

Query 2: What Should be Patched First?

Query 2: *What should be patched first?*

Finds the CVEs with an open incident in the SOC log and a fix not yet applied, and keeps only those that ThreatGraph reports as used in known ransomware campaigns.

It combines all three sources: open incidents (*SOC*), fix not done (*patch office*), ransomware use (*ThreatGraph*).

Results are ordered by risk (CVSS score, highest first), then by planned date.

```
SELECT ?cve ?product ?owner ?planned ?score
WHERE {
  ?i a soc:Incident ; soc:status "open" ; soc:exploits ?v .   # SOC incident log
  ?p pt:forCve ?v ; pt:owner_team ?owner ;                   # patch office
     pt:patched "no" ; pt:planned_date ?planned .
  ?v dct:identifier ?cve ; tg:product ?product ;             # ThreatGraph
     tg:ransomwareUse "known" ;
     tg:nvdRecord/tg:cvssV31/tg:baseScore ?score .
}
ORDER BY DESC(?score) ?planned
```

CVE	Product	Owner	Planned	CVSS
CVE-2023-48788	FortiClient EMS	Network	2025-08-06	**9.8**
CVE-2017-5638	Struts	Desktop	2025-09-05	**9.8**
CVE-2021-27065	Exchange Server	Network	2025-09-15	**7.8**

Interoperability Example: Results

Key results

A company can combine JSON-LD with its own incident log and patch tracker through the CVE address, and get a verified answer: which exploited vulnerabilities to patch first.

DIP

In OAIS terms, the API and the JSON-LD export are dissemination packages: users read them without knowing **ThreatGraph** database.

Limitations

The vocabulary for the project’s own terms is still a placeholder, and the CVE address must be written in the same way by every source.

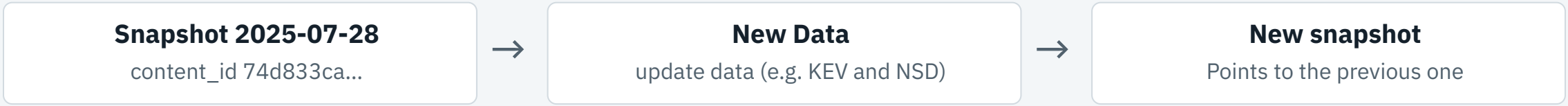
Snapshots: Integrity and Verifiable Version

A snapshot is a frozen, verifiable copy of the dataset at one instant. Sources keep changing, so a snapshot lets anyone repeat an analysis on exactly the same data. In OAIS terms, each snapshot is an AIP.

Content-based ID
The content_id is the SHA-256 of the table hashes. Same data, same ID, whatever the label or the time.

Never overwritten
A snapshot is written completely or not at all, and then made read-only. An existing snapshot is never overwritten.

Provenance inside
The manifest records the source versions, the table hashes and the previous snapshot (prov:wasRevisionOf).



Limitation (persistence): A content hash identifies content, not location: it is unique, but not persistent. Persistence needs someone who commits to keep it and a system that resolves it, like DOI or Handle. The hashes also detect accidental changes, not a deliberate rewrite of both data and hashes.

Snapshots: Example PHP2Shell

The database is frozen at 28 July 2025. Two recent **PHP2Shell** vulnerabilities (CVE-2026-109618 and CVE-2026-109619) are then added (CISA KEV and NVD), and the snapshot must show exactly this change.

Baseline

Snapshot 2025-07-28: 9 tables, 1,391 vulnerabilities (KEV and NVD)

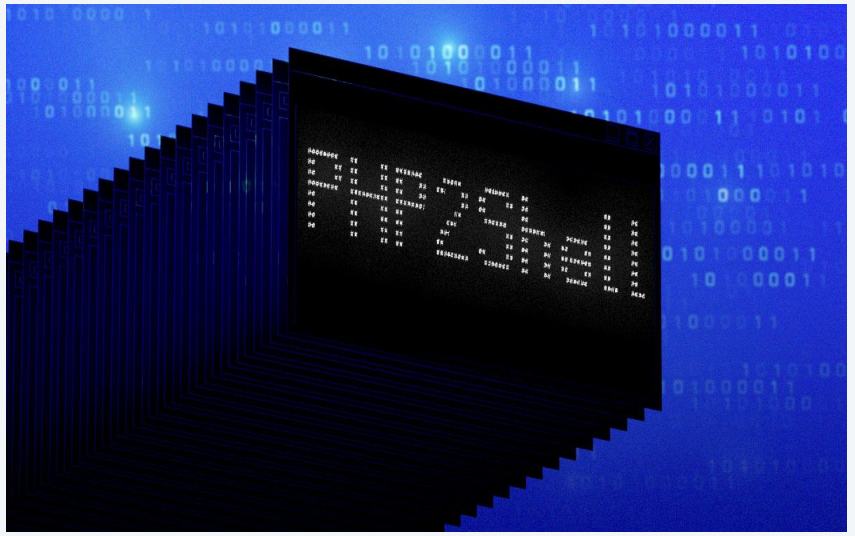
Append new CVEs

Two new CVEs are added: 2 rows in KEV and 2 rows in NVD.

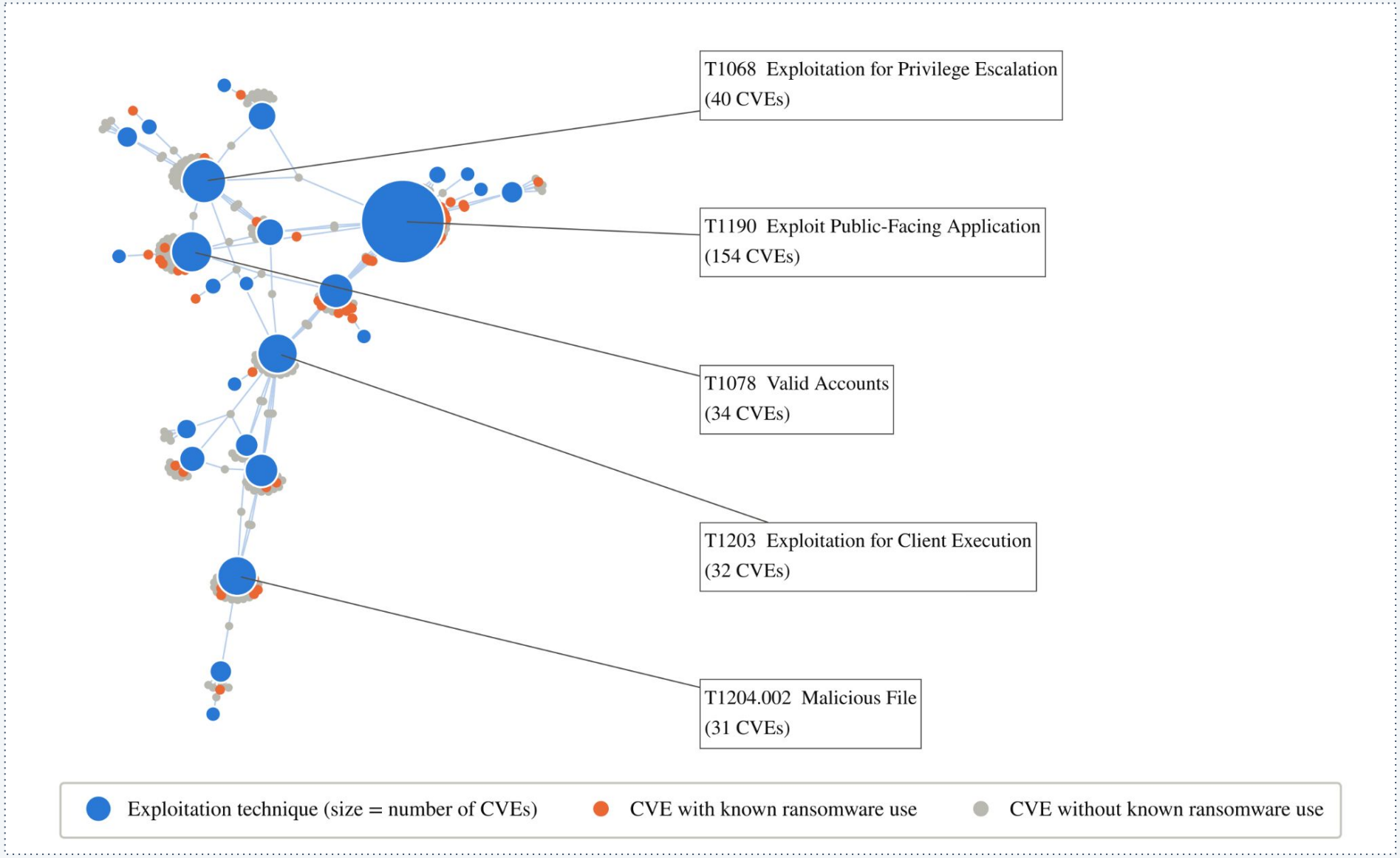
New snapshot

Same format, new added data. The new snapshot points to the old one.

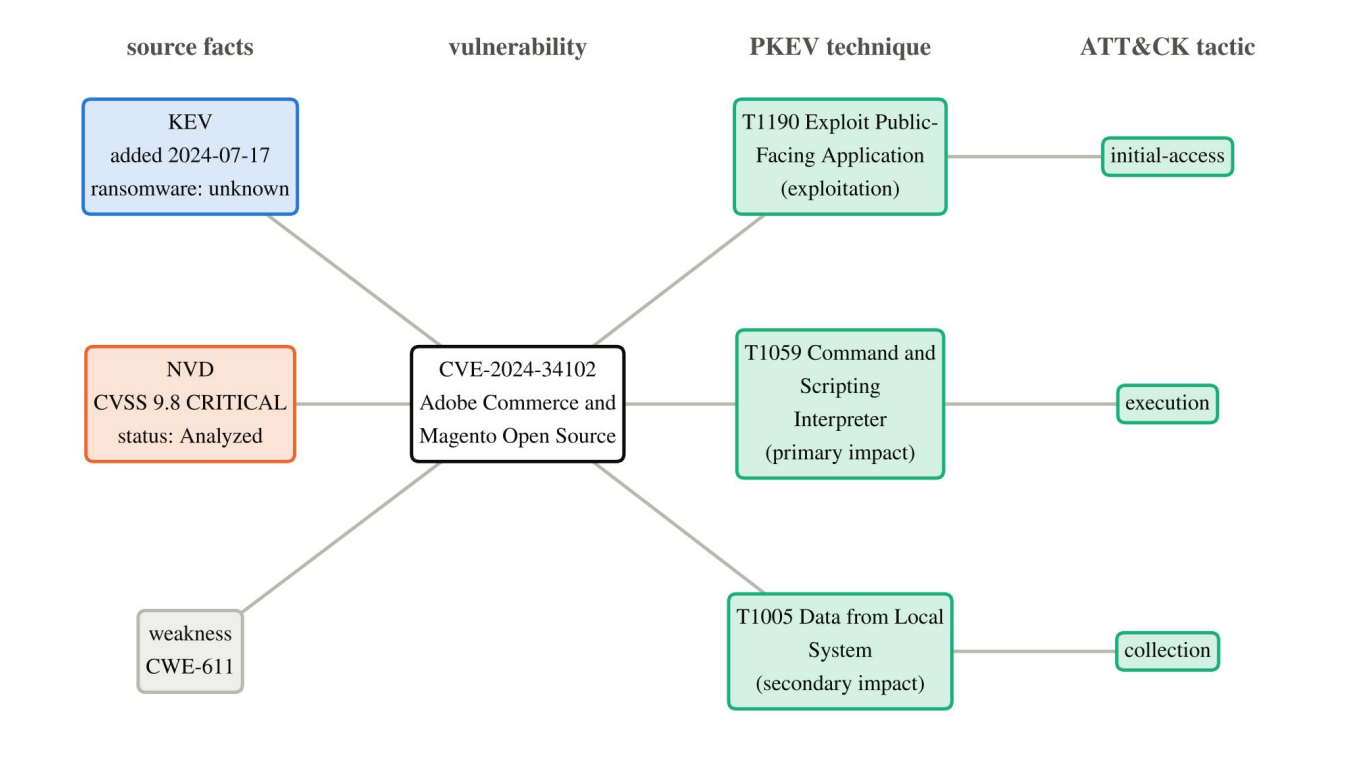
Table	Rows A	Rows B	SHA-256 A (2025-07-28)	SHA-256 B (2026-10-26)	Result
kev_vulnerability	**1,391**	**1,393**	**fc0c9410db47…**	**c8d0c507c673…**	**CHANGED**
kev_cwe	1,328	1,328	b84bc6036a20…	b84bc6036a20…	same
nvd_cve	**1,391**	**1,393**	**435b99c03249…**	**1e56dc4e9825…**	**CHANGED**
nvd_cve_cwe	2,523	2,523	51ec143965c2…	51ec143965c2…	same
attack_tactic	14	14	7e2e881702e4…	7e2e881702e4…	same
attack_technique	799	799	912ea6914222…	912ea6914222…	same
attack_technique_tactic	1,051	1,051	3008af112e5a…	3008af112e5a…	same
pkev_mapping	1,183	1,183	6b76edf9253c…	6b76edf9253c…	same
pkev_mapping_reference	2,256	2,256	c9111e106ef3…	c9111e106ef3…	same
content_id (whole snapshot)			**74d833caed5f…**	**870c1ec4b0ef…**	**CHANGED**



Graph visualization: Most used exploitation techniques in KEV



Case study: One CVE, Four Sources



CVE-2024-34102 (Adobe Commerce and Magento Open Source) is one node. Each source adds a different kind of fact; the CVE id and the technique id link them.

- **KEV:** exploited, added on 2024-07-17
- **NVD:** CVSS 9.8, CRITICAL
- **PKEV:** how it is exploited and what it leads to (T1190 → T1059 → T1005)
- **ATT&CK:** the attacker goals: initial access, execution, collection

ThreatGraph and FAIR principles assessment

ID	Principle	Status	How we treat it, and the honest limit
F1	Persistent identifiers	◐ Partial	Public CVE, CWE and ATT&CK IRIs. The project's namespace is a placeholder.
F2	Rich metadata	✓ Done	DCAT and PROV description of the dataset.
F3	Metadata name the data	✓ Done	URL and SHA-256 of each distributions.
F4	Searchable registry	✕ Not done	No registry, DOI or repository deposit.
A1	Retrievable by identifier	◐ Partial	HTTP API, JSON and JSON-LD. Local server only.
A1.1	Open, free protocol	✓ Done	HTTP is open and free.
A1.2	Authentication where needed	- Not needed	All data are public; no authentication is implemented.
A2	Metadata outlive the data	✕ Not done	No external registry keeps the metadata after the data are gone.
I1	Formal language	✓ Done	JSON-LD, readable as RDF database, SPARQL for queries.
I2	FAIR vocabularies	◐ Partial	DCAT, PROV, DCTERMS. Own terms use a placeholder namespace.
I3	Qualified references	✓ Done	Links to CWE and ATT&CK IRIs; the mapping is its own node.
R1	Rich, relevant attributes	✓ Done	KEV, NVD, ATT&CK and PKEV fields, with typed relations.
R1.1	Usage licence	✓ Done	Recorded per source.
R1.2	Provenance	✓ Done	Pinned source versions, SHA-256, cleaning log, snapshots linked with PROV.
R1.3	Community standards	✓ Done	Standard identifiers and vocabularies.

Answer to Research Question

Question

“How can heterogeneous open data sources be integrated into one database so that every link between them is traceable, versioned and reusable?”

Answer

Heterogeneous open sources can be integrated by pinning each source to a hashed version, cleaning it with recorded rules, linking records through public identifiers (CVE id, technique id), and freezing every version as a hashed snapshot linked to the previous one. Traceability comes from the manifests, versioning from the snapshots, reusability from open formats and DCAT/PROV.

Key Results & Conclusions

OAIS: a clear path from submission to dissemination

Four open sources are integrated into 9 related tables, linked by shared identifiers (CVE id, technique id). The four downloads are the SIPs, the snapshot is the AIP, and the API with the JSON-LD export is the DIP.

Integrity and provenance

A SHA-256 hash per table and one content_id per snapshot make every version verifiable. In a simulated update, adding two vulnerabilities changed only the 2 affected tables, as confirmed by comparing their hashes. Each snapshot also records its sources, its cleaning code and its previous version.

Interoperability and working example

JSON-LD, RDF and SPARQL allowed third-party data to be merged with ThreatGraph through the shared CVE identifier, without changing the database. A working example with two other datasets (an incident log and a patch tracker) shows interoperability in practice.

FAIR principles and project strengths

The FAIR principles were assessed by connecting the course theory to the practical project. The strengths are a reproducible pipeline, open standards (CSV, JSON-LD, SPARQL) and vocabularies (DCAT, PROV).

Limitations & Future Work

Main Limitations

- **Mapping Coverage**: PKEV maps only 30% of KEV vulnerabilities, so every ATT&CK result describes this subset.
- **Frozen Data**: the database reflects one snapshot dated 28 July 2025, so recent vulnerabilities are missing.
- **Source quality**: the results depend on the accuracy and the selection criteria of the sources.
- **Local deployment**: the API runs on Django's development server with SQLite. This is fine for a demo, but not for many concurrent users.
- **Placeholder vocabulary**: the tg: namespace is not yet a real published vocabulary so terms are not fully FAIR.

Future Work

- **Persistent identifiers**: publish the custom vocabulary at a stable address and assign a persistent identifier (e.g., a DOI).
- **Searchable registry**: register dataset in a public catalogue so that users can find it.
- **Lasting metadata**: deposit metadata in an external repository so that it outlives the server.
- **Updates**: ingest new KEV and NVD releases and new PKEV mappings as new snapshots, linked to the previous ones.
- **Public API**: host the API on a public server with a production web server.

References

- **CISA, KEV Catalog:** *cisa.gov/resources-tools/resources/kev-catalog · github.com/cisagov/kev-data*
- **MITRE ATT&CK (STIX data):** *github.com/mitre-attack/attack-stix-data*
- **CTID, Mappings Explorer (PKEV):** *github.com/center-for-threat-informed-defense/mappings-explorer*
- **NVD:** *github.com/fkie-cad/nvd-json-data-feeds*
- **OAIS reference model:** ISO 14721:2012 · CCSDS 650.0-M-2 · course Lecture 23
- **FAIR principles:** *Wilkinson et al., Scientific Data, 2016*
- **W3C:** JSON-LD 1.1 · DCAT · PROV-O

Thanks for listening